Clean Code and OOP Principles

Code blocks and explanations from the Kotlin Recipe Search project

Package structure • Inheritance • Abstract class • Interface • Polymorphism • Enum • Companion object • Error handling • Collections

Teacher: Yusif Yusifov
Student: Zeynab Davudzada
Date: 18.06.2026
Subject: OOP

Project: final-2026-Alulaaustralis-bit

Project structure and clean code approach

The code is divided into packages so each part has a separate responsibility.

```
src/main/kotlin/
├── app/
│   └── Main.kt                    → proqramın giriş nöqtəsi
├── model/
│   ├── Recipe.kt                  → abstract baza class
│   ├── VegetarianRecipe.kt        → Recipe-dən miras alır
│   ├── NonVegetarianRecipe.kt     → Recipe-dən miras alır
│   ├── BakingRecipe.kt            → Recipe + CookingMethod
│   └── CuisineType.kt             → enum + companion object
├── interfaces/
│   └── CookingMethod.kt           → bişirmə vaxtı müqaviləsi
└── service/
    └── RecipeSearch.kt            → axtarış məntiqi
```

Why is package structure important?

In the project, code is separated by function: the app package contains the program entry point, the model package stores recipe objects, the interfaces package defines contract-like behavior, and the service package keeps the search logic. This structure supports clean code because Main.kt does not take all responsibilities, each class has one clear role, and new recipe types can be added more easily later.

Abstract class: Recipe.kt

A shared base model is created for all recipe types.

CODE BLOCK

```kotlin
package model

abstract class Recipe(
    private var name: String,
    private var ingredients: List<String>,
    private var instructions: String
) {
    fun getName(): String = name
    fun setName(value: String) { name = value }

    fun getIngredients(): List<String> = ingredients
    fun setIngredients(value: List<String>) { ingredients = value }

    fun getInstructions(): String = instructions
    fun setInstructions(value: String) { instructions = value }

    abstract fun describe(): String
}
```

EXPLANATION / ANALYSIS

What is used in this code?

Recipe is an abstract class and works as the main model of the project. name, ingredients, and instructions are kept private; this is an example of encapsulation. Getter and setter methods provide controlled access to the data. Because describe() is an abstract function, Recipe does not give the full behavior directly; it only defines a common rule that subclasses must implement.

Inheritance: VegetarianRecipe and NonVegetarianRecipe

Subclasses inherit from Recipe and override only the part that is different.

CODE BLOCK

```
class VegetarianRecipe(
    name: String,
    ingredients: List<String>,
    instructions: String
) : Recipe(name, ingredients, instructions) {

    override fun describe(): String {
        return "Vegetarian Recipe: ${getName()}"
    }
}

class NonVegetarianRecipe(
    name: String,
    ingredients: List<String>,
    instructions: String
) : Recipe(name, ingredients, instructions) {

    override fun describe(): String {
        return "Non-Vegetarian Recipe: ${getName()}"
    }
}
```

EXPLANATION / ANALYSIS

How does inheritance work?

VegetarianRecipe and NonVegetarianRecipe inherit from the Recipe class. The name, ingredients, and instructions values received in the constructor are passed to the base class. This approach reduces repeated code. Each subclass overrides the describe() method according to its own recipe type, so the same base structure is kept while the output becomes different.

Interface: CookingMethod and BakingRecipe

An interface gives a class an additional behavior rule.

CODE BLOCK

```kotlin
package interfaces

interface CookingMethod {
    fun get_cooking_time(): Double
}

class BakingRecipe(
    name: String,
    ingredients: List<String>,
    instructions: String,
    private var ovenTemp: Double = DEFAULT_OVEN_TEMP,
    private var bakeTime: Double = DEFAULT_BAKE_TIME
) : Recipe(name, ingredients, instructions), CookingMethod {

    override fun get_cooking_time(): Double = bakeTime
}
```

EXPLANATION / ANALYSIS

Interface usage

The CookingMethod interface is a contract that returns the cooking time for a recipe. BakingRecipe both inherits from the Recipe class and implements the CookingMethod interface. In Kotlin, a class can inherit from only one class, but interfaces can be used to add extra behavior. For this reason, BakingRecipe is both a general recipe and a class that returns cooking time information.

Polymorphism: one List<Recipe>, different objects

The code calls describe() without checking the real type of the object.

CODE BLOCK

```
val recipesByCuisine: Map<CuisineType, List<Recipe>> = mapOf(
    CuisineType.AZERBAIJANI to listOf(
        VegetarianRecipe("Piti", listOf("noxud", "kartof"), "Saxsı qabda bişirin."),
        NonVegetarianRecipe("Dövgə", listOf("quzu", "noxud"), "Bişirib qarışdırın."),
        BakingRecipe("Paxlava", listOf("xəmir", "qoz"), "Sobada bişir.", 180.0, 40.0)
    )
)

search.getAllRecipes().forEach {
    println(it.describe())
}
```

EXPLANATION / ANALYSIS

Where is polymorphism visible?

List<Recipe> is used inside recipesByCuisine, but the list stores VegetarianRecipe, NonVegetarianRecipe, and BakingRecipe objects together. When search.getAllRecipes().forEach { println(it.describe()) } is executed, each object runs its own overridden describe() method. This means that even if the variable type is Recipe, the behavior of the real class is executed at runtime.

Enum: CuisineType.kt

Cuisine types are stored as limited and safe choices.

CODE BLOCK

```kotlin
enum class CuisineType {
    AZERBAIJANI,
    ITALIAN,
    CHINESE,
    AMERICAN,
    OTHER;

    fun displayName(): String = when (this) {
        AZERBAIJANI -> "Azerbaijan"
        ITALIAN -> "Italy"
        CHINESE -> "China"
        AMERICAN -> "America"
        OTHER -> "Other"
    }
}
```

EXPLANATION / ANALYSIS

Why was enum chosen?

The CuisineType enum class stores the available cuisine types as a fixed list: AZERBAIJANI, ITALIAN, CHINESE, AMERICAN, and OTHER. Using enum instead of string values reduces the risk of typos and invalid values. The displayName() function converts the enum name into a readable form for the user.

Companion object: choice conversion and default values

Class-level functions and constants are stored in companion objects.

CODE BLOCK

```
companion object {
    fun fromChoice(choice: String?): CuisineType {
        return when (choice?.trim()) {
            "1" -> AZERBAIJANI
            "2" -> ITALIAN
            "3" -> CHINESE
            "4" -> AMERICAN
            "5" -> OTHER
            else -> AZERBAIJANI
        }
    }
}

companion object {
    const val DEFAULT_OVEN_TEMP = 180.0
    const val DEFAULT_BAKE_TIME = 40.0
}
```

EXPLANATION / ANALYSIS

Role of the companion object

CuisineType.fromChoice() is inside a companion object, so it can be called without creating a separate CuisineType object. This function converts user choices into enum values. In BakingRecipe, DEFAULT_OVEN_TEMP and DEFAULT_BAKE_TIME are stored as const val. This reduces the use of magic numbers and centralizes default values.

Error handling: null-safety and default choice

There is no try-catch in the project, but input mistakes are handled safely.

CODE BLOCK

```
fun selectCuisine(): CuisineType {
    print("Choice: ")
    return CuisineType.fromChoice(readLine())
}

fun fromChoice(choice: String?): CuisineType {
    return when (choice?.trim()) {
        "1" -> AZERBAIJANI
        "2" -> ITALIAN
        "3" -> CHINESE
        "4" -> AMERICAN
        "5" -> OTHER
        else -> AZERBAIJANI
    }
}

val name = readLine().orEmpty()
```

EXPLANATION / ANALYSIS

How are errors handled?

Because readLine() can return null, the code considers the String? type. choice?.trim() does not stop the program when the value is null. readLine().orEmpty() converts null input into an empty string. The else -> AZERBAIJANI part in fromChoice() returns a default value when the selection is wrong or empty. This prevents the program from crashing because of simple input mistakes.

Collections: Map, List, and MutableList

Recipes are grouped, stored, and searched using collections.

CODE BLOCK

```kotlin
private val recipes: MutableList<Recipe> = mutableListOf()

fun addRecipe(recipe: Recipe) {
    recipes.add(recipe)
}

fun getAllRecipes(): List<Recipe> = recipes.toList()

fun search_by_ingredient(ingredient: String): List<Recipe> {
    return recipes.filter { recipe ->
        recipe.getIngredients().any { it.contains(ingredient, ignoreCase = true) }
    }
}
```

EXPLANATION / ANALYSIS

Where are collections used?

Map<CuisineType, List<Recipe>> groups recipes by cuisine type. MutableList<Recipe> is used inside RecipeSearch because recipes are added and removed. getAllRecipes() returns recipes.toList(); this does not allow the internal mutable list to be changed from outside. The filter and any functions show functional use of the Kotlin collection API.

Service layer: RecipeSearch.kt

The search logic is separated from Main.kt and stored in a separate service class.

CODE BLOCK

```kotlin
class RecipeSearch {
    private val recipes: MutableList<Recipe> = mutableListOf()

    fun clearRecipes() {
        recipes.clear()
    }

    fun search_by_name(name: String): List<Recipe> {
        return recipes.filter {
            it.getName().contains(name, ignoreCase = true)
        }
    }
}
```

EXPLANATION / ANALYSIS

Separation of concerns

The RecipeSearch class is responsible for storing, clearing, and searching recipes. This helps Main.kt focus only on the menu and user flow. The search_by_name() method selects matching recipes with filter, and ignoreCase=true avoids problems with uppercase/lowercase differences. This structure improves readability and extensibility.

Summary: main principles used in the project

The code examples are taken from the real Kotlin files of the project.

```kotlin
var currentCuisine = selectCuisine()
val search = RecipeSearch()

fun loadCuisine(cuisine: CuisineType) {
    search.clearRecipes()
    recipesByCuisine[cuisine]?.forEach { search.addRecipe(it) }
    println("
Loaded: ${cuisine.displayName()}
")
}

loadCuisine(currentCuisine)
```

Strengths of the project

- Package structure divides the code into app, model, interfaces, and service parts.
- The Recipe abstract class creates a shared base behavior.
- VegetarianRecipe, NonVegetarianRecipe, and BakingRecipe are built with inheritance.
- The CookingMethod interface keeps additional behavior separate.
- Because of polymorphism, different objects work together inside List<Recipe>.
- Enum and companion object make choices and default values safer.
- Collections and null-safety help the program work simply and stably.